ASSIGNMENT 2 – CO2

C++ Functions and Object-Oriented Concepts

Expanded Assignment Report

Name: Busireddy Chaithanya Prakash Reddy

Register Number: 192425255

Programme: B.Tech – Artificial Intelligence and Machine Learning

Course: C++

Assignment Overview

CO2 Coverage: Design C++ programs by applying functions and object-oriented concepts, including function prototyping, call by reference, return by reference, inline functions, default arguments, function overloading, class specification, member functions, access specifiers, arrays within a class, static data members and member functions, arrays of objects, and friend functions.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: 31/10/2026

Bloom's Level: K5 – Evaluate. Each task asks the student to examine a design, predict behaviour, test it, and justify a decision.

Mandatory Execution and GitHub Submission

- Execute every task individually, including all Complete the Code programs.
- Create a repository named DSA01-<RegisterNumber>-Assignment2 and share it with the course faculty. It may be public or private with the faculty added as a collaborator.
- Create one folder for each task, such as Task01, Task02, and so on. Store C++ source files, input files, and output screenshots in the relevant folder.
- Include README.md with name, register number, compiler and version, exact compilation command, run instructions, and a short result note for each task.
- Commit progressively, with at least one commit for each task. A single upload at the end is not accepted by the assignment instructions.
- Include the GitHub repository link on the cover page and in the submission field.

How to Complete the Evidence

For Predict the Output, write the predicted output before executing the program. Run it afterward and explain any difference using the C++ rules involved. For Complete the Code, copy the given program, replace every TODO without changing the rest of the program, compile, execute, and compare with the expected output. For Alternative Concept and Complexity Analysis, explain how the design would change under the alternative approach and compare time and space using Big-O notation.

Contents

- Q1 – Call by Value, by Reference and by Pointer
- Q2 – Return by Reference and Its Dangers
- Q3 – Default Arguments and Function Overloading
- Q4 – Specifying a Class with an Array Inside It
- Q5 – Static Members and Arrays of Objects
- Q6 – Friend Functions and Inline Functions

- Q7 – Diagnose, Redesign and Validate
- Submission Checklist

Q1 – Call by Value, by Reference and by Pointer (6 Marks)

Problem Context: Hostel Fee Concession Updater

The hostel office applies fee concessions such as merit (10%), sports (15%), and single-parent concessions (25%). A function may display a changed local fee while the original value in the caller remains unchanged. The task is to compare call by value, call by reference, and call by pointer.

Concept Explanation

Call by value passes a copy of the argument. Changes to the parameter affect only that copy. Call by reference gives the function an alias for the original variable, so changes are visible to the caller. Call by pointer passes an address; dereferencing the pointer allows the function to update the original object.

```
#include <iostream>
using namespace std;

void byValue(long long fee, int percent) {
    fee -= fee * percent / 100;
}

void byReference(long long& fee, int percent) {
    fee -= fee * percent / 100;
}

void byPointer(long long* fee, int percent) {
    if (fee != nullptr)
        *fee -= (*fee) * percent / 100;
}

int main() {
    long long a = 45000, b = 45000, c = 45000;
    byValue(a, 10);
    byReference(b, 10);
    byPointer(&c, 10);
    cout << a << '\n' << b << '\n' << c << '\n';
    return 0;
}
```

Expected Result and Interpretation

```
45000
40500
40500
```

The value passed to byValue remains 45000 because only the local parameter changes. The reference and pointer versions update the caller's fee to 40500. The pointer version checks for nullptr before dereferencing, which avoids dereferencing a null pointer.

Predict the Output: Given Program

```
#include <iostream>

void change(int a, int& b, int* c) {
   a += 10;
   b += 10;
   *c += 10;
}
int& bigger(int& p, int& q) { return p > q ? p : q; }
void show(int x, int y = 5, int z = 10) {
   std::cout << x + y + z << ' ';
}
int main() {
   int a = 1, b = 2, c = 3;
   change(a, b, &c);
   std::cout << a << ' ' << b << ' ' << c << '\n';
   bigger(a, c) = 100;
   std::cout << a << ' ' << c << '\n';
   show(1);
   show(1, 2);
   show(1, 2, 3);
   std::cout << '\n';
}
```

Predicted output (write this before running):

```
1 12 13
100 13
16 13 6
```

Explanation: a is passed by value to change(), so it remains 1. b is passed by reference and becomes 12. c is passed by pointer and becomes 13. The bigger() function returns a reference to the larger argument; at that moment c (13) is larger than a (1), so the assignment sets c to 100. Therefore, the correct second line is actually '1 100'. The show() calls use default values as follows: show(1) uses y=5 and z=10; show(1,2) uses z=10; show(1,2,3) uses no defaults.

Verified predicted output after correcting the reference selection:

```
1 12 13
1 100
16 13 6
```

Complete the Code – Call by Reference

```
void applyConcession(long long& fee, int percent) {
   fee -= fee * percent / 100;
}
```

Expected output from the supplied program:

```
after call by value : 45000
after call by reference: 40500
```

Alternative Design and Complexity

A return-value design can keep the original unchanged until the caller assigns the result: long long applyConcession(long long fee, int percent) { return fee - fee * percent / 100; }. The call becomes hostelFee = applyConcession(hostelFee, 10). All three approaches take O(1) time and O(1) auxiliary space for a long long. For a large object, passing by value may copy the object, while

reference and const reference avoid that copy. Use const reference when a large object must be read but not modified.

Conclusion

References are suitable when the function is intended to update the caller's fee. Returning a new value is also clear and avoids hidden mutation. The choice should be documented and consistent.

Q2 – Return by Reference and Its Dangers (4 Marks)

Problem Context: Examination Marks Register

A marks register stores marks in an array. Returning a reference to the highest mark allows the caller to modify the original array element directly. Returning a reference to a local variable is unsafe because that local variable ceases to exist when the function returns.

Implementation

```
#include <iostream>
using namespace std;

int& highestMark(int marks[], int n) {
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[best]) best = i;
    return marks[best];
}

// Do not use this function: it returns a dangling reference.
// int& unsafeValue() {
//     int local = 10;
//     return local; // warning: reference to local variable
// }

int main() {
    int marks[] = {72, 88, 81, 95, 90};
    highestMark(marks, 5) += 2;
    for (int mark : marks) cout << mark << ' ';
    cout << '\n';
}
```

Output and Explanation

```
72 88 81 97 90
```

highestMark() returns a reference to marks[3], the highest mark. The expression highestMark(marks, 5) += 2 therefore modifies the original array element. A reference to a local variable becomes dangling after return; using it causes undefined behaviour and must be avoided.

Reference, Const Reference, Copy, Index and Pointer

- int&: allows the caller to modify the referenced array element.
- const int&: allows reading without copying but prevents modification through that reference.
- int: returns an independent copy; modifying the returned value does not change the array.

- int index: returns the position of the maximum. The caller can use marks[index] to update the value.
- int*: returns an address. The caller must ensure that the pointed-to element remains valid.

Vector Growth Condition

When a std::vector grows beyond its capacity, it may reallocate storage. References, pointers, and iterators to its elements can then become invalid. An index is generally safer across reallocation, provided the vector's logical contents and size are still checked before using the index.

Complexity Analysis

Finding the maximum requires scanning n marks, so time complexity is O(n). The algorithm uses O(1) auxiliary space. Returning a reference, pointer, or index does not change the O(n) search time. An index avoids a dangling element reference caused solely by vector reallocation, but it must still be within the current vector bounds.

Conclusion

Return by reference is useful for direct modification of existing objects, but the referenced object must outlive the reference. Never return a reference to an ordinary local variable.

Q3 – Default Arguments and Function Overloading (6 Marks)

Problem Context: Courier Charge Calculator

A courier calculator charges by parcel weight, with express delivery and insurance as optional services. Overlapping overloads and default arguments can make a call ambiguous when more than one candidate can accept the same arguments.

Redesigned Example

```
#include <iostream>
using namespace std;

double charge(double weightKg, bool express = false,
              bool insured = false) {
    double total = weightKg * 50.0;
    if (express) total += 40.0;
    if (insured) total += 20.0;
    return total;
}

double charge(int weightGrams) {
    return charge(weightGrams / 1000.0, false, false);
}

struct Parcel {
    double weightKg;
    bool express;
    bool insured;
};

double charge(const Parcel& p) {
```

```
    return charge(p.weightKg, p.express, p.insured);
}

int main() {
    cout << charge(2) << '\n';
    cout << charge(2.0) << '\n';
    cout << charge(2.0f) << '\n';
    cout << charge(1500) << '\n';
    cout << charge(2.0, true) << '\n';
    cout << charge(Parcel{3.0, true, true}) << '\n';
}
```

Predicted Calls and Output

The integer literal 2 selects charge(int), while 2.0 selects charge(double, bool, bool) using defaults. The float literal 2.0f can convert to double and selects the double-weight overload in this design. 1500 selects the int overload and is interpreted as grams. charge(2.0, true) uses the default insured=false. A Parcel argument selects charge(const Parcel&).

```
charge(2)                       -> 0.1
charge(2.0)                     -> 100
charge(2.0f)                    -> 100
charge(1500)                    -> 75
charge(2.0, true)               -> 140
charge(Parcel{3.0, true, true})   -> 210
```

These results follow the example's stated pricing rule: ₹50 per kilogram, plus ₹40 for express and ₹20 for insurance. The charge(int) overload treats its integer argument as grams, so 2 grams costs ₹0.10 and 1500 grams costs ₹75. For real applications, distinct names such as chargeGrams() and chargeKg() can make units clearer.

Important Rules

- An exact match is generally preferred over a promotion, and a promotion is generally preferred over a standard conversion.
- Default arguments supply missing trailing parameters; they do not create a separate overload.
- Default arguments must be provided from the right, because a call cannot omit an earlier parameter while supplying a later one.
- Avoid declaring several overloads that can all accept the same call through defaults and conversions.

Alternative Design and Complexity

A single Parcel structure can group weight and optional services in one object. This makes adding new options easier because the function signature may remain unchanged. Overload resolution occurs at compile time and adds no runtime cost. Passing a small numeric value is O(1) time and space; passing a large structure by const reference avoids copying it.

Conclusion

Use overloads when input types or meanings differ, and use default arguments for genuinely optional trailing settings. Make units explicit to reduce accidental misuse.

Q4 – Specifying a Class with an Array Inside It (6 Marks)

Problem Context: Savings Account Mini-Statement

Each SavingsAccount object stores its account number, balance, and most recent ten transactions. The array is private so that outside code cannot change balances or transaction history without following the class's rules.

Program

```
#include <iostream>
#include <string>
using namespace std;

class SavingsAccount {
    string accountNumber;
    double balance;
    string transactions[10];
    int count;

    void record(const string& entry) {
        if (count < 10) {
            transactions[count++] = entry;
        } else {
            for (int i = 1; i < 10; ++i)
                transactions[i - 1] = transactions[i];
            transactions[9] = entry;
        }
    }

public:
    SavingsAccount(string number, double opening)
        : accountNumber(number), balance(opening), count(0) {
        record("Opening balance: " + to_string(opening));
    }

    void deposit(double amount);
    void withdraw(double amount);
    void miniStatement() const;
};

void SavingsAccount::deposit(double amount) {
    if (amount <= 0) return;
    balance += amount;
    record("Deposit: " + to_string(amount));
}

void SavingsAccount::withdraw(double amount) {
    if (amount <= 0 || amount > balance) return;
    balance -= amount;
    record("Withdrawal: " + to_string(amount));
}

void SavingsAccount::miniStatement() const {
    cout << "Account: " << accountNumber << '\n';
    cout << "Balance: " << balance << '\n';
    cout << "Recent transactions:\n";
    for (int i = 0; i < count; ++i)
        cout << transactions[i] << '\n';
}
```

```
int main() {
    SavingsAccount account("SB1001", 5000);
    account.deposit(1000);
    account.withdraw(750);
    account.miniStatement();
}
```

Design Explanation

- Private data protects the balance and transaction history from direct external modification.
- Public member functions define the permitted operations and maintain the account's rules.
- Some functions are declared inside the class, while deposit(), withdraw(), and miniStatement() are defined outside using SavingsAccount::.
- When the array is full, shifting discards the oldest transaction and appends the newest one.

Alternative: Circular Buffer or std::deque

Shifting k entries takes O(k) time for each new transaction after the buffer is full. A circular buffer stores a start index and overwrites the oldest slot in O(1) time. A std::deque can use push_back() and pop_front() to maintain a bounded history, typically with O(1) operations at the ends. A fixed array uses O(10) storage; changing the limit to 50 mainly changes the capacity and related loop boundaries.

Conclusion

A private array provides a simple bounded history. A circular buffer is a strong choice when transaction volume grows and constant-time insertion is important.

Q5 – Static Members and Arrays of Objects (7 Marks)

Problem Context: College Bus Seat Tickets

The seat counter belongs to the entire BusTicket class, not to an individual ticket. A static data member is shared by every object. A static member function can access static members directly but does not have a this pointer and cannot directly access a particular object's non-static members.

Complete the Code

```
#include <iostream>
#include <string>
using namespace std;

class BusTicket {
    static int issued;
    static const int CAPACITY = 3;
    int seat;
    string name;

public:
    explicit BusTicket(const string& n) : seat(0), name(n) {
        if (issued < CAPACITY) {
            ++issued;
            seat = issued;
```

```
        }
    }

    static int count() {
        return issued;
    }

    void print() const {
        cout << name << ": "
             << (seat ? "Seat " + to_string(seat) : string("Waiting"))
             << '\n';
    }
};

int BusTicket::issued = 0;

int main() {
    BusTicket trip[4] = {
        BusTicket("Asha"), BusTicket("Bala"),
        BusTicket("Chitra"), BusTicket("Dev")
    };
    for (const BusTicket& t : trip) t.print();
    cout << "Issued: " << BusTicket::count() << '\n';
}
```

Expected Output

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

Predict the Output: Static Token

```
#include <iostream>
class Token {
    static int next;
    int id;
public:
    Token() { id = ++next; }
    int get() const { return id; }
    static int issued() { return next - 100; }
};
int Token::next = 100;
int main() {
    Token counter[3];
    Token extra;
    std::cout << counter[0].get() << ' '
              << counter[2].get() << ' '
              << extra.get() << ' '
              << Token::issued() << '\n';
}
```

Predicted output:

```
101 103 104 4
```

The static variable starts at 100 because it is explicitly initialized by the out-of-class definition. The constructor runs four times: three times for counter[3] and once for extra. Each constructor increments the shared counter.

Alternative Design and Complexity

A global counter can work but is less well encapsulated. A separate TicketOffice object allows independent counters for different buses and makes the design more flexible, but that office must be passed to ticket creation operations. All approaches use O(1) time per ticket and O(1) counter space per bus or class. A single static counter is not suitable if two buses need independent seat numbering.

Conclusion

Static members are appropriate for class-wide state shared by all objects. Use a separate office or counter object when multiple independent buses must be represented.

Q6 – Friend Functions and Inline Functions (8 Marks)

Problem Context: Train Journey Planner

The Time class stores hours and minutes privately. Two free functions compare times and calculate the interval between them. A friend declaration allows these functions to access private data without making the data public.

Completed Program

```
#include <iostream>
#include <cstdlib>
using namespace std;

class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};

inline int minutesBetween(const Time& a, const Time& b) {
    int first = a.h * 60 + a.m;
    int second = b.h * 60 + b.m;
    return abs(first - second);
}

inline bool earlier(const Time& a, const Time& b) {
    if (a.h != b.h) return a.h < b.h;
    return a.m < b.m;
}

int main() {
    Time departure(9, 45), arrival(13, 10);
    cout << minutesBetween(departure, arrival) << ' '
         << boolalpha << earlier(arrival, departure) << '\n';
    Time sameA(12, 0), sameB(12, 0);
    cout << minutesBetween(sameA, sameB) << ' '
         << earlier(sameA, sameB) << '\n';
}
```

Expected Output

```
205 false
0 false
```

9:45 is 585 minutes after midnight and 13:10 is 790 minutes after midnight, so the difference is 205 minutes. arrival is not earlier than departure, so the comparison is false. Equal times have a difference of zero, and neither is earlier than the other.

Friend and Member Functions

- A friend function is not a member function; it is called like an ordinary function and may access private members only because the class grants friendship.
- A member function is called through an object and can access that object's private members directly.
- Public getters preserve private storage but expose read-only access through a controlled interface.
- The inline keyword permits a function definition to appear in a header and suggests that the compiler may substitute the function body at a call site. It does not guarantee substitution.

Timing Investigation

Compile and measure a loop of 10,000,000 calls with -O0 and -O2, first with the functions declared inline and then with inline removed. Use the same input values and ensure the return values are used so the compiler cannot discard all work. Record actual timings from your compiler; do not invent measurement results. At -O2, the compiler may inline functions regardless of the keyword, so the difference can be small or absent.

```
g++ -std=c++17 -O0 time_test.cpp -o time_test_O0
g++ -std=c++17 -O2 time_test.cpp -o time_test_O2
./time_test_O0
./time_test_O2
```

Conclusion

Friend functions are reasonable when an operation naturally compares two objects equally. If encapsulation is better served by member functions or getters, use those alternatives. Measure performance rather than assuming inline always improves speed.

Q7 – Diagnose, Redesign and Validate (13 Marks)

This design challenge asks you to select one program from Q1–Q6 that has a significant limitation as data grows or requirements change. The following worked example uses the transaction history from Q4. Replace the sample observations with your own program output and measured evidence before submission.

A. Diagnose the Limitation (2 Marks)

Original design: the fixed array stores ten transactions. Once full, every new transaction shifts the remaining nine entries to the left before writing the newest entry. This maintains the correct history, but insertion after the buffer fills takes O(k) time for k stored entries.

B. Define the Design Objective (2 Marks)

Objective: preserve only the latest ten transactions while reducing the insertion operation after the buffer is full from O(k) shifting to O(1) overwrite time. The design must still print transactions from oldest to newest.

C. Design the Revised Solution (4 Marks)

Use a circular buffer with a fixed array, a start index indicating the oldest entry, and a count indicating the number of stored transactions. When the buffer is not full, append at (start + count) modulo capacity. When full, overwrite the oldest entry and advance start modulo capacity.

```
Pseudocode:
record(newTransaction):
    if count < capacity:
        position = (start + count) % capacity
        data[position] = newTransaction
        count = count + 1
    else:
        data[start] = newTransaction
        start = (start + 1) % capacity

printHistory():
    for i from 0 to count - 1:
        position = (start + i) % capacity
        print data[position]
```

Extra memory is minimal: the fixed array, start index, and count. The trade-off is slightly more complicated indexing. A deque is another option, but it may involve dynamic allocation and is not required to understand the original limitation.

D. Predict Before Implementing (1 Mark)

Prediction: after the buffer fills, each new transaction will be recorded with a constant number of assignments and index updates. The printed history will remain in chronological order and contain no more than ten transactions.

E. Implement and Validate (2 Marks)

Run the original and revised versions using the same sequence of at least 15 transactions. Confirm that both display exactly the last ten transactions. Measure the time needed to add a large number of transactions, repeat the test several times, and report actual results.

F. Defend the Final Design (2 Marks)

The circular buffer meets the objective if the same most-recent-ten history is retained and insertion time no longer depends on shifting all stored elements. The original shifting array may remain

preferable for very small histories where straightforward code is more important than insertion efficiency.

Comparison Template

Fill the following measurements after executing both versions. Values below are intentionally left blank so actual evidence can be entered.

Number of transactions tested: _____

Original version total insertion time: _____

Circular-buffer version total insertion time: _____

Both versions retain the same latest ten entries? _____

Compiler and optimization flags: _____

Conclusion

A successful redesign is not simply a replacement with a library feature. It identifies the original limitation, states a measurable goal, implements a suitable alternative, and validates the result under equivalent conditions.

Submission Checklist

- Cover page includes name, register number, assignment title, and GitHub repository link.
- Q1–Q6 include program source, predictions written before execution, actual output, explanation, alternative design, and complexity analysis where requested.
- Q5 includes static-member prediction and the completed BusTicket program output.
- Q6 includes actual timing measurements for -O0 and -O2; no timing values are fabricated.
- Q7 includes the diagnosed limitation, measurable objective, redesign, prediction, original-versus-redesigned test evidence, and final evaluation.
- GitHub repository is named DSA01-<RegisterNumber>-Assignment2 and contains separate task folders.
- Each task folder includes source code and output screenshots; README.md documents compiler, commands, run steps, and results.
- At least one progressive commit is made for each task.
- The working GitHub link is included in the report and submitted to the faculty.

General References / Execution Notes

Use a C++17-compatible compiler such as g++. The source assignment gives example compilation flags including -std=c++17 -Wall -Wextra. Warnings should be read and understood, particularly for returning references to local variables. Include your own execution screenshots and do not claim that a program was executed unless you have run it.